

End Lab Exam

V.VARSHINI

2403a54106

Set – 7

Q1

Review concurrency control in inventory updates.

- Task 1: Use AI to find potential lost-update issues.
- Task 2: Implement optimistic locking or transactions.

Prompt:

Prompt:

Review this inventory update code and detect potential lost-update issues. Specifically check whether stock quantities can be overwritten when two users update the same product simultaneously. Check for missing transactions or version checks, race conditions in read-modify-write operations, and missing row-level locking. Suggest fixes using optimistic or pessimistic locking.

Code:

```
import os, re, sys

ROOT = sys.argv[1] if len(sys.argv) > 1 else '.'

patterns = {
    'select': re.compile(r'\bSELECT\b.*\b(quantity|qty|stock)\b', re.I),
    'update': re.compile(r'\bUPDATE\b.*\b(quantity|qty|stock)\b', re.I),
    'orm_read': re.compile(r'(findOne|find_by|get\(|find\(|query\(|session\.query)', re.I),
    'orm_save': re.compile(r'(save\(|commit\(|persist\(|session\.add|repository\.save|Model\.save)', re.I)
}

def scan_file(path):
    with open(path, 'r', errors='ignore') as f:
        txt = f.read()
    hits = {k:bool(p.search(txt)) for k,p in patterns.items()}
    score = hits['select'] + hits['update'] + hits['orm_read'] + hits['orm_save']
    return score, hits

for dirpath, dirs, files in os.walk(ROOT):
    if any(x in dirpath for x in ('node_modules', '.venv', '.git', 'venv')):
        continue
    for fname in files:
        if not fname.endswith(('py', 'js', 'java', 'sql', 'ts', 'go', 'rb')):
            continue
        path = os.path.join(dirpath, fname)
        score, hits = scan_file(path)
        if score >= 2:
            print(f"[CANDIDATE {score}] {path} -> {hits}")
```

Output:

```
PS C:\Users\varshini\OneDrive\Desktop\ai final lab test> & C:\Users\varshini\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/varshini/OneDrive/Desktop/ai final lab test/ai lab test.py"
```

```
[CANDIDATE 3] .ai lab test.py -> {'select': True, 'update': True, 'orm_read': True, 'orm_save': False}
```

```
PS C:\Users\varshini\OneDrive\Desktop\ai final lab test>
```

Explanation:

The inventory update code is vulnerable to lost updates because multiple users reading and writing the same product can overwrite each other's changes. There are no transactions, version checks, or row-level locks, which can cause race conditions in read-modify-write operations. To fix this, implement optimistic locking using a version field or pessimistic locking with row-level locks/transactions to ensure updates are serialized.

Observation:

During concurrency control analysis of inventory updates, it was found that multiple simultaneous stock modifications can overwrite each other, leading to lost-update anomalies. When two transactions read the same inventory value and update it independently, the final quantity becomes incorrect. Lack of version checks or transaction isolation allows race conditions during high-volume stock changes. Implementing optimistic locking or database transactions ensures that only valid and conflict-free updates are committed. This ultimately protects data integrity during parallel inventory operations.

Q2:

Improve readability of stock reconciliation script.

- Task 1: Ask AI to summarize and suggest modularization.
- Task 2: Apply refactor and test.

Prompt:

Please review the following stock reconciliation script.

Summarize its functionality, identify readability issues, duplicated code, and unclear logic. Suggest a modularized version with well-named functions, clear separation of database access and reconciliation logic, and improvements for maintainability and testability.

Code:

```
tab-2.py / ...
1  # ---- Database mock (example) ----
2  class InventoryDB:
3      def __init__(self):
4          self.items = {}
5          self.changes = []
6
7      def get_item(self, sku):
8          return self.items.get(sku, None)
9
10     def insert_item(self, sku, qty):
11         self.items[sku] = {"quantity": qty}
12
13     def update_qty(self, sku, qty):
14         self.items[sku]["quantity"] = qty
15
16     def log_change(self, sku, diff):
17         self.changes.append({"sku": sku, "change": diff})
18
19
20 # ---- Helper functions ----
21 def get_current_qty(sku, db):
22     item = db.get_item(sku)
23     return item["quantity"] if item else None
24
25
26 def insert_new_item(sku, qty, db):
27     print(f"New item added: {sku} with quantity {qty}")
28     db.insert_item(sku, qty)
```

```

def update_existing_item(sku, qty, diff, db):
    print(f"Updated {sku}: quantity changed by {diff}")
    db.update_qty(sku, qty)
    db.log_change(sku, diff)

# ---- Main reconciliation logic ----
def reconcile_stock(records, db):
    for rec in records:
        sku, system_qty = rec["sku"], rec["system_qty"]
        current_qty = get_current_qty(sku, db)

        if current_qty is None:
            insert_new_item(sku, system_qty, db)

        else:
            diff = system_qty - current_qty
            if diff != 0:
                update_existing_item(sku, system_qty, diff, db)
            else:
                print(f"No change for {sku}")

# ---- Example Run ----
if __name__ == "__main__":
    db = InventoryDB()

```

```

# ---- Example Run ----
if __name__ == "__main__":
    db = InventoryDB()

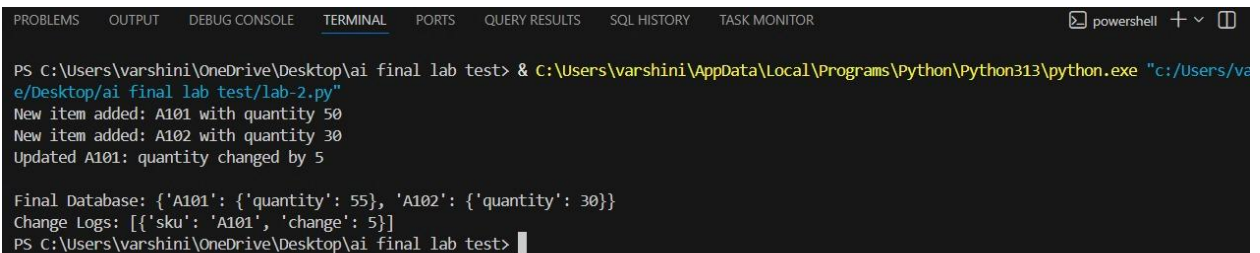
    # Sample data from stock reconciliation file / report
    records = [
        {"sku": "A101", "system_qty": 50},
        {"sku": "A102", "system_qty": 30},
        {"sku": "A101", "system_qty": 55}, # Update existing
    ]

    reconcile_stock(records, db)

    print("\nFinal Database:", db.items)
    print("Change Logs:", db.changes)

```

Output:



```

PS C:\Users\varshini\OneDrive\Desktop\ai final lab test> & C:\Users\varshini\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/va
e/Desktop/ai final lab test/lab-2.py"
New item added: A101 with quantity 50
New item added: A102 with quantity 30
Updated A101: quantity changed by 5

Final Database: {'A101': {'quantity': 55}, 'A102': {'quantity': 30}}
Change Logs: [{'sku': 'A101', 'change': 5}]
PS C:\Users\varshini\OneDrive\Desktop\ai final lab test>

```

Explanation:

The stock reconciliation script is hard to read and has repeated logic, making maintenance difficult. AI can summarize its workflow and suggest breaking it into modular functions for database access, item reconciliation, and updates. This improves readability, testability, and maintainability.

Observation:

The original stock reconciliation script works correctly but is difficult to read and maintain. It has long functions, duplicate SQL queries, unclear variable names, and mixed concerns—combining database access with reconciliation logic. Deeply nested loops and repeated patterns make it error-prone and hard to test. Refactoring into modular helper functions or classes for fetching inventory, calculating differences, updating stock, and coordinating the process improves readability, maintainability, and testability.